

Repository Code Listing

BinaryHeap.cpp

```
1  #include "BinaryHeap.h"
2
3  void BinaryHeap::Clear()
4  {
5      key.clear();
6      pos.clear();
7      satellite.clear();
8  }
9
10 void BinaryHeap::Insert(double k, int s)
11 {
12     //Adjust the structures to fit new data
13     if(s >= (int)pos.size())
14     {
15         pos.resize(s+1, -1);
16         key.resize(s+1);
17         //Recall that position 0 of satellite is unused
18         satellite.resize(s+2);
19     }
20     //If satellite is already in the heap
21     else if(pos[s] != -1)
22     {
23         throw "Error: satellite already in heap";
24     }
25
26     int i;
27     for(i = ++size; i/2 > 0 && GREATER(key[satellite[i/2]], k); i /=
28 2)
29     {
30         satellite[i] = satellite[i/2];
31         pos[satellite[i]] = i;
32     }
33     satellite[i] = s;
34     pos[s] = i;
35     key[s] = k;
36 }
37
38 int BinaryHeap::Size()
39 {
40     return size;
41 }
42
43 int BinaryHeap::DeleteMin()
44 {
45     if(size == 0)
46         throw "Error: empty heap";
47
48     int min = satellite[1];
49     int slast = satellite[size--];
50
51
52     int child;
53     int i;
54     for(i = 1, child = 2; child <= size; i = child, child *= 2)
```

```

    {
55         if(child < size && GREATER(key[satellite[child]],
56 key[satellite[child+1]]))
57             child++;
58
59         if(GREATER(key[slast], key[satellite[child]]))
60         {
61             satellite[i] = satellite[child];
62             pos[satellite[child]] = i;
63         }
64         else
65             break;
66     }
67     satellite[i] = slast;
68     pos[slast] = i;
69
70     pos[min] = -1;
71
72     return min;
73 }
74
75 void BinaryHeap::ChangeKey(double k, int s)
76 {
77     Remove(s);
78     Insert(k, s);
79 }
80
81 void BinaryHeap::Remove(int s)
82 {
83     int i;
84     for(i = pos[s]; i/2 > 0; i /= 2)
85     {
86         satellite[i] = satellite[i/2];
87         pos[satellite[i]] = i;
88     }
89     satellite[1] = s;
90     pos[s] = 1;
91
92     DeleteMin();
93 }

```

BinaryHeap.h

```

1  #pragma once
2
3  #include "Globals.h"
4  #include <vector>
5  using namespace std;
6
7  /*
8  This is a binary heap for pairs of the type (double key, int satellite)
9  It is assumed that satellites are unique integers
10 This is the case with graph algorithms, in which satellites are vertex or
11 edge indices
12 */
13 class BinaryHeap
14 {
15 public:
16     BinaryHeap(): satellite(1), size(0) {};
17

```

```

18         //Inserts (key k, satellite s) in the heap
19         void Insert(double k, int s);
20         //Deletes the element with minimum key and returns its satellite
21         information
22         int DeleteMin();
23         //Changes the key of the element with satellite s
24         void ChangeKey(double k, int s);
25         //Removes the element with satellite s
26         void Remove(int s);
27         //Returns the number of elements in the heap
28         int Size();
29         //Resets the structure
30         void Clear();
31     private:
32         vector<double> key;//Given the satellite, this is its key
33         vector<int> pos;//Given the satellite, this is its position in
34         the heap
35         vector<int> satellite;//This is the heap!
36
37         //Number of elements in the heap
38         int size;
39     };

```

Example.cpp

```

1  #include "Matching.h"
2  #include <fstream>
3  #include "Graph.h"
4  #include <cstdlib>
5  #include <iostream>
6  #include <string>
7  #include <sstream>
8  using namespace std;
9
10 pair< Graph, vector<double> > CreateRandomGraph()
11 {
12     //random seed
13     int x;
14     cin >> x;
15     srand( x );
16
17     //Please see Graph.h for a description of the interface
18     int n = 50;
19
20     Graph G(n);
21     vector<double> cost;
22     for(int i = 0; i < n; i++)
23         for(int j = i+1; j < n; j++)
24             if(rand()%10 == 0)
25             {
26                 G.AddEdge(i, j);
27                 cost.push_back(rand()%1000);
28             }
29
30     return make_pair(G, cost);
31 }
32
33 Graph ReadGraph(string filename)
34 {

```

```

35 //Please see Graph.h for a description of the interface
36
37 ifstream file;
38 file.open(filename.c_str());
39
40 string s;
41 getline(file, s);
42 stringstream ss(s);
43 int n;
44 ss >> n;
45 getline(file, s);
46 ss.str(s);
47 ss.clear();
48 int m;
49 ss >> m;
50
51 Graph G(n);
52 for(int i = 0; i < m; i++)
53 {
54     getline(file, s);
55     ss.str(s);
56     ss.clear();
57     int u, v;
58     ss >> u >> v;
59
60     G.AddEdge(u, v);
61 }
62
63 file.close();
64 return G;
65 }
66
67 pair< Graph, vector<double> > ReadWeightedGraph(string filename)
68 {
69     //Please see Graph.h for a description of the interface
70
71     ifstream file;
72     file.open(filename.c_str());
73
74     string s;
75     getline(file, s);
76     stringstream ss(s);
77     int n;
78     ss >> n;
79     getline(file, s);
80     ss.str(s);
81     ss.clear();
82     int m;
83     ss >> m;
84
85     Graph G(n);
86     vector<double> cost(m);
87     for(int i = 0; i < m; i++)
88     {
89         getline(file, s);
90         ss.str(s);
91         ss.clear();
92         int u, v;
93         double c;
94         ss >> u >> v >> c;
95
96         G.AddEdge(u, v);
97         cost[G.GetEdgeIndex(u, v)] = c;
98     }

```

```

99
100     file.close();
101     return make_pair(G, cost);
102 }
103
104 void MinimumCostPerfectMatchingExample(string filename)
105 {
106     Graph G;
107     vector<double> cost;
108
109     //Read the graph
110     pair< Graph, vector<double> > p = ReadWeightedGraph(filename);
111     //pair< Graph, vector<double> > p = CreateRandomGraph();
112     G = p.first;
113     cost = p.second;
114
115     //Create a Matching instance passing the graph
116     Matching M(G);
117
118     //Pass the costs to solve the problem
119     pair< list<int>, double > solution =
120 M.SolveMinimumCostPerfectMatching(cost);
121
122     list<int> matching = solution.first;
123     double obj = solution.second;
124
125     cout << "Optimal matching cost: " << obj << endl;
126     cout << "Edges in the matching:" << endl;
127     for(list<int>::iterator it = matching.begin(); it !=
128 matching.end(); it++)
129     {
130         pair<int, int> e = G.GetEdge( *it );
131
132         cout << e.first << " " << e.second << endl;
133     }
134 }
135
136 void MaximumMatchingExample(string filename)
137 {
138     Graph G = ReadGraph(filename);
139     Matching M(G);
140
141     list<int> matching;
142     matching = M.SolveMaximumMatching();
143
144     cout << "Number of edges in the maximum matching: " <<
145 matching.size() << endl;
146     cout << "Edges in the matching:" << endl;
147     for(list<int>::iterator it = matching.begin(); it !=
148 matching.end(); it++)
149     {
150         pair<int, int> e = G.GetEdge( *it );
151
152         cout << e.first << " " << e.second << endl;
153     }
154 }
155
156 int main(int argc, char* argv[])
157 {
158     string filename = "";
159     string algorithm = "";
160
161     int i = 1;
162     while(i < argc)

```

```

    {
        string a(argv[i]);
        if(a == "-f")
            filename = argv[++i];
        else if(a == "--minweight")
            algorithm = "minweight";
        else if(a == "--max")
            algorithm = "max";
163
164        i++;
165    }
166
167    if(filename == "" || algorithm == "")
168    {
169        cout << "usage: ./example -f <filename> <--minweight |
170 --max>" << endl;
171        cout << "--minweight for minimum weight perfect
172 matching" << endl;
173        cout << "--max for maximum cardinality matching" <<
174 endl;
175        cout << "file format:" << endl;
176        cout <<
177 "the first two lines give n (number of vertices) and m (number of
178 edges)," << endl;
179        cout <<
180 "followed by m lines, each with a tuple (u, v [, c]) representing the
181 edges," << endl;
182        cout << "where u and v are the endpoints (0-based
183 indexing) of the edge and c is its cost" << endl;
184        cout << "the cost is optional if --max is specified" <<
185 endl;
186        return 1;
187    }
188
189    try
190    {
191        if(algorithm == "minweight")
192            MinimumCostPerfectMatchingExample(filename);
193        else
194            MaximumMatchingExample(filename);
195    }
196    catch(const char * msg)
197    {
        cout << msg << endl;
        return 1;
    }

    return 0;
}

```

Globals.h

```

1 #pragma once
2
3 #define EPSILON 0.000001
4 #define INFINITO 1000000000.0
5 #define GREATER(A, B) ((A) - (B) > EPSILON)
6 #define LESS(A, B) ((B) - (A) > EPSILON)
7 #define EQUAL(A, B) (fabs((A) - (B)) < EPSILON)
8 #define GREATER_EQUAL(A, B) (GREATER((A),(B)) || EQUAL((A),(B)))
9 #define LESS_EQUAL(A, B) (LESS((A),(B)) || EQUAL((A),(B)))

```

```

10 #define MIN(A, B) (LESS((A),(B)) ? (A) : (B))
11 #define MAX(A, B) (LESS((A),(B)) ? (B) : (A))

```

Graph.cpp

```

1  #include "Graph.h"
2
3  Graph::Graph(int n, const list< pair<int, int> > & edges):
4      n(n),
5      m(edges.size()),
6      adjMat(n, vector<bool>(n, false)),
7      adjList(n),
8      edges(),
9      edgeIndex(n, vector<int>(n, -1))
10 {
11     for(list< pair<int, int> >::const_iterator it = edges.begin();
12 it != edges.end(); it++)
13     {
14         int u = (*it).first;
15         int v = (*it).second;
16
17         AddEdge(u, v);
18     }
19 }
20
21 pair<int, int> Graph::GetEdge(int e) const
22 {
23     if(e > (int)edges.size())
24         throw "Error: edge does not exist";
25
26     return edges[e];
27 }
28
29 int Graph::GetEdgeIndex(int u, int v) const
30 {
31     if( u > n or
32        v > n )
33         throw "Error: vertex does not exist";
34
35     if(edgeIndex[u][v] == -1)
36         throw "Error: edge does not exist";
37
38     return edgeIndex[u][v];
39 }
40
41 void Graph::AddVertex()
42 {
43     for(int i = 0; i < n; i++)
44     {
45         adjMat[i].push_back(false);
46         edgeIndex[i].push_back(-1);
47     }
48     n++;
49     adjMat.push_back( vector<bool>(n, false) );
50     edgeIndex.push_back( vector<int>(n, -1) );
51     adjList.push_back( list<int>() );
52 }
53
54 void Graph::AddEdge(int u, int v)
55 {

```

```

56         if( u > n or
57             v > n )
58             throw "Error: vertex does not exist";
59
60         if(adjMat[u][v]) return;
61
62         adjMat[u][v] = adjMat[v][u] = true;
63         adjList[u].push_back(v);
64         adjList[v].push_back(u);
65
66         edges.push_back(pair<int, int>(u, v));
67         edgeIndex[u][v] = edgeIndex[v][u] = m++;
68     }
69
70     const list<int> & Graph::AdjList(int v) const
71     {
72         if(v > n)
73             throw "Error: vertex does not exist";
74
75         return adjList[v];
76     }
77
78     const vector< vector<bool> > & Graph::AdjMat() const
79     {
80         return adjMat;
81     }

```

Graph.h

```

1  #pragma once
2
3  #include <list>
4  #include <vector>
5  using namespace std;
6
7  class Graph
8  {
9  public:
10     //n is the number of vertices
11     //edges is a list of pairs representing the edges (default =
12     empty list)
13     Graph(int n, const list< pair<int, int> > & edges = list<
14     pair<int, int> >());
15
16     //Default constructor creates an empty graph
17     Graph(): n(0), m(0) {};
18
19     //Returns the number of vertices
20     int GetNumVertices() const { return n; };
21     //Returns the number of edges
22     int GetNumEdges() const { return m; };
23
24     //Given the edge's index, returns its endpoints as a pair
25     pair<int, int> GetEdge(int e) const;
26     //Given the endpoints, returns the index
27     int GetEdgeIndex(int u, int v) const;
28
29     //Adds a new vertex to the graph
30     void AddVertex();
31     //Adds a new edge to the graph

```

```

32         void AddEdge(int u, int v);
33         //Returns the adjacency list of a vertex
34         const list<int> & AdjList(int v) const;
35
36         //Returns the graph's adjacency matrix
37         const vector< vector<bool> > & AdjMat() const;
38     private:
39         //Number of vertices
40         int n;
41         //Number of edges
42         int m;
43
44         //Adjacency matrix
45         vector< vector<bool> > adjMat;
46
47         //Adjacency lists
48         vector< list<int> > adjList;
49
50         //Array of edges
51         vector< pair<int, int> > edges;
52
53         //Indices of the edges
54         vector< vector<int> > edgeIndex;
55     };

```

Matching.cpp

```

1  #include "Matching.h"
2
3  Matching::Matching(const Graph & G):
4      G(G),
5      outer(2*G.GetNumVertices()),
6      deep(2*G.GetNumVertices()),
7      shallow(2*G.GetNumVertices()),
8      tip(2*G.GetNumVertices()),
9      active(2*G.GetNumVertices()),
10     type(2*G.GetNumVertices()),
11     forest(2*G.GetNumVertices()),
12     root(2*G.GetNumVertices()),
13     blocked(2*G.GetNumVertices()),
14     dual(2*G.GetNumVertices()),
15     slack(G.GetNumEdges()),
16     mate(2*G.GetNumVertices()),
17     m(G.GetNumEdges()),
18     n(G.GetNumVertices()),
19     visited(2*G.GetNumVertices())
20 {
21 }
22
23 void Matching::Grow()
24 {
25     Reset();
26
27     //All unmatched vertices will be roots in a forest that will be
28     grown
29     //The forest is grown by extending a unmatched vertex w through
30     a matched edge u-v in a BFS fashion
31     while(!forestList.empty())
32     {

```

```

33         int w = outer[forestList.front()];
34         forestList.pop_front();
35
36         //w might be a blossom
37         //we have to explore all the connections from vertices
38         inside the blossom to other vertices
39         for(list<int>::iterator it = deep[w].begin(); it !=
40         deep[w].end(); it++)
41         {
42             int u = *it;
43
44             int cont = false;
45             for(list<int>::const_iterator jt =
46             G.AdjList(u).begin(); jt != G.AdjList(u).end(); jt++)
47             {
48                 int v = *jt;
49
50                 if(IsEdgeBlocked(u, v)) continue;
51
52                 //u is even and v is odd
53                 if(type[outer[v]] == ODD)
54                     continue;
55
56                 //if v is unlabeled
57                 if(type[outer[v]] != EVEN)
58                 {
59                     //We grow the alternating forest
60                     int vm = mate[outer[v]];
61
62                     forest[outer[v]] = u;
63                     type[outer[v]] = ODD;
64                     root[outer[v]] = root[outer[u]];
65                     forest[outer[vm]] = v;
66                     type[outer[vm]] = EVEN;
67                     root[outer[vm]] =
68                     root[outer[u]];
69
70                     if(!visited[outer[vm]])
71                     {
72
73                         forestList.push_back(vm);
74
75                         visited[outer[vm]] =
76                         true;
77                     }
78                     //If v is even and u and v are on
79                     different trees
80                     //we found an augmenting path
81                     else if(root[outer[v]] !=
82                     root[outer[u]])
83                     {
84                         Augment(u,v);
85                         Reset();
86
87                         cont = true;
88                         break;
89                     }
90                     //If u and v are even and on the same
91                     tree
92                     //we found a blossom
93                     else if(outer[u] != outer[v])
94                     {
95                         int b = Blossom(u,v);
96

```

```

97         forestList.push_front(b);
98         visited[b] = true;
99
100         cont = true;
101         break;
102     }
103 }
104     if(cont) break;
105 }
106 }
107
108 //Check whether the matching is perfect
109 perfect = true;
110 for(int i = 0; i < n; i++)
111     if(mate[outer[i]] == -1)
112         perfect = false;
113 }
114
115 bool Matching::IsAdjacent(int u, int v)
116 {
117     return (G.AdjMat()[u][v] and not IsEdgeBlocked(u, v));
118 }
119
120 bool Matching::IsEdgeBlocked(int u, int v)
121 {
122     return GREATER(slack[ G.GetEdgeIndex(u, v) ], 0);
123 }
124
125 bool Matching::IsEdgeBlocked(int e)
126 {
127     return GREATER(slack[e], 0);
128 }
129
130 //Vertices will be selected in non-decreasing order of their degree
131 //Each time an unmatched vertex is selected, it is matched to its
132 adjacent unmatched vertex of minimum degree
133 void Matching::Heuristic()
134 {
135     vector<int> degree(n, 0);
136     BinaryHeap B;
137
138     for(int i = 0; i < m; i++)
139     {
140         if(IsEdgeBlocked(i)) continue;
141
142         pair<int, int> p = G.GetEdge(i);
143         int u = p.first;
144         int v = p.second;
145
146         degree[u]++;
147         degree[v]++;
148     }
149
150     for(int i = 0; i < n; i++)
151         B.Insert(degree[i], i);
152
153     while(B.Size() > 0)
154     {
155         int u = B.DeleteMin();
156         if(mate[outer[u]] == -1)
157         {
158             int min = -1;
159             for(list<int>::const_iterator it =
160 G.AdjList(u).begin(); it != G.AdjList(u).end(); it++)

```

```

161         {
162             int v = *it;
163
164             if(IsEdgeBlocked(u, v) or
165                (outer[u] == outer[v]) or
166                (mate[outer[v]] != -1) )
167                 continue;
168
169             if(min == -1 or degree[v] < degree[min])
170                 min = v;
171         }
172         if(min != -1)
173         {
174             mate[outer[u]] = min;
175             mate[outer[min]] = u;
176         }
177     }
178 }
179 }
180
181 //Destroys a blossom recursively
182 void Matching::DestroyBlossom(int t)
183 {
184     if((t < n) or
185        (blocked[t] and GREATER(dual[t], 0))) return;
186
187     for(list<int>::iterator it = shallow[t].begin(); it !=
188 shallow[t].end(); it++)
189     {
190         int s = *it;
191         outer[s] = s;
192         for(list<int>::iterator jt = deep[s].begin(); jt !=
193 deep[s].end(); jt++)
194             outer[*jt] = s;
195
196         DestroyBlossom(s);
197     }
198
199     active[t] = false;
200     blocked[t] = false;
201     AddFreeBlossomIndex(t);
202     mate[t] = -1;
203 }
204
205 void Matching::Expand(int u, bool expandBlocked = false)
206 {
207     int v = outer[mate[u]];
208
209     int index = m;
210     int p = -1, q = -1;
211     //Find the regular edge {p,q} of minimum index connecting u and
212     its mate
213     //We use the minimum index to grant that the two possible
214     blossoms u and v will use the same edge for a mate
215     for(list<int>::iterator it = deep[u].begin(); it !=
216 deep[u].end(); it++)
217     {
218         int di = *it;
219         for(list<int>::iterator jt = deep[v].begin(); jt !=
220 deep[v].end(); jt++)
221         {
222             int dj = *jt;
223             if(IsAdjacent(di, dj) and G.GetEdgeIndex(di,
224 dj) < index)

```

```

225         {
226             index = G.GetEdgeIndex(di, dj);
227             p = di;
228             q = dj;
229         }
230     }
231 }
232
233     mate[u] = q;
234     mate[v] = p;
235     //If u is a regular vertex, we are done
236     if(u < n or (blocked[u] and not expandBlocked)) return;
237
238     bool found = false;
239     //Find the position t of the new tip of the blossom
240     for(list<int>::iterator it = shallow[u].begin(); it !=
241 shallow[u].end() and not found; )
242     {
243         int si = *it;
244         for(list<int>::iterator jt = deep[si].begin(); jt !=
245 deep[si].end() and not found; jt++)
246         {
247             if(*jt == p )
248                 found = true;
249         }
250         it++;
251         if(not found)
252         {
253             shallow[u].push_back(si);
254             shallow[u].pop_front();
255         }
256     }
257
258     list<int>::iterator it = shallow[u].begin();
259     //Adjust the mate of the tip
260     mate[*it] = mate[u];
261     it++;
262     //
263     //Now we go through the odd circuit adjusting the new mates
264     while(it != shallow[u].end())
265     {
266         list<int>::iterator itnext = it;
267         itnext++;
268         mate[*it] = *itnext;
269         mate[*itnext] = *it;
270         itnext++;
271         it = itnext;
272     }
273
274     //We update the sets blossom, shallow, and outer since this
275 blossom is being deactivated
276     for(list<int>::iterator it = shallow[u].begin(); it !=
277 shallow[u].end(); it++)
278     {
279         int s = *it;
280         outer[s] = s;
281         for(list<int>::iterator jt = deep[s].begin(); jt !=
282 deep[s].end(); jt++)
283             outer[*jt] = s;
284     }
285     active[u] = false;
286     AddFreeBlossomIndex(u);
287
288     //Expand the vertices in the blossom

```

```

289         for(list<int>::iterator it = shallow[u].begin(); it !=
290 shallow[u].end(); it++)
291             Expand(*it, expandBlocked);
292     }
293 }
294
295 //Augment the path root[u], ..., u, v, ..., root[v]
296 void Matching::Augment(int u, int v)
297 {
298     //We go from u and v to its respective roots, alternating the
299 matching
300     int p = outer[u];
301     int q = outer[v];
302     int outv = q;
303     int fp = forest[p];
304     mate[p] = q;
305     mate[q] = p;
306     Expand(p);
307     Expand(q);
308     while(fp != -1)
309     {
310         q = outer[forest[p]];
311         p = outer[forest[q]];
312         fp = forest[p];
313
314         mate[p] = q;
315         mate[q] = p;
316         Expand(p);
317         Expand(q);
318     }
319
320     p = outv;
321     fp = forest[p];
322     while(fp != -1)
323     {
324         q = outer[forest[p]];
325         p = outer[forest[q]];
326         fp = forest[p];
327
328         mate[p] = q;
329         mate[q] = p;
330         Expand(p);
331         Expand(q);
332     }
333 }
334
335 void Matching::Reset()
336 {
337     for(int i = 0; i < 2*n; i++)
338     {
339         forest[i] = -1;
340         root[i] = i;
341
342         if(i >= n and active[i] and outer[i] == i)
343             DestroyBlossom(i);
344     }
345
346     visited.assign(2*n, 0);
347     forestList.clear();
348     for(int i = 0; i < n; i++)
349     {
350         if(mate[outer[i]] == -1)
351         {
352             type[outer[i]] = 2;

```

```

353             if(!visited[outer[i]])
354                 forestList.push_back(i);
355             visited[outer[i]] = true;
356         }
357         else type[outer[i]] = 0;
358     }
359 }
360
361 int Matching::GetFreeBlossomIndex()
362 {
363     int i = free.back();
364     free.pop_back();
365     return i;
366 }
367
368 void Matching::AddFreeBlossomIndex(int i)
369 {
370     free.push_back(i);
371 }
372
373 void Matching::ClearBlossomIndices()
374 {
375     free.clear();
376     for(int i = n; i < 2*n; i++)
377         AddFreeBlossomIndex(i);
378 }
379
380 //Contracts the blossom w, ..., u, v, ..., w, where w is the first
381 vertex that appears in the paths from u and v to their respective roots
382 int Matching::Blossom(int u, int v)
383 {
384     int t = GetFreeBlossomIndex();
385
386     vector<bool> isInPath(2*n, false);
387
388     //Find the tip of the blossom
389     int u_ = u;
390     while(u_ != -1)
391     {
392         isInPath[outer[u_]] = true;
393         u_ = forest[outer[u_]];
394     }
395
396     int v_ = outer[v];
397     while(not isInPath[v_])
398         v_ = outer[forest[v_]];
399     tip[t] = v_;
400
401     //Find the odd circuit, update shallow, outer, blossom and deep
402     //First we construct the set shallow (the odd circuit)
403     list<int> circuit;
404     u_ = outer[u];
405     circuit.push_front(u_);
406     while(u_ != tip[t])
407     {
408         u_ = outer[forest[u_]];
409         circuit.push_front(u_);
410     }
411
412     shallow[t].clear();
413     deep[t].clear();
414     for(list<int>::iterator it = circuit.begin(); it !=
415 circuit.end(); it++)

```

```

417         {
418             shallow[t].push_back(*it);
419         }
420
421         v_ = outer[v];
422         while(v_ != tip[t])
423         {
424             shallow[t].push_back(v_);
425             v_ = outer[forest[v_]];
426         }
427
428         //Now we construct deep and update outer
429         for(list<int>::iterator it = shallow[t].begin(); it !=
430 shallow[t].end(); it++)
431         {
432             u_ = *it;
433             outer[u_] = t;
434             for(list<int>::iterator jt = deep[u_].begin(); jt !=
435 deep[u_].end(); jt++)
436             {
437                 deep[t].push_back(*jt);
438                 outer[*jt] = t;
439             }
440         }
441
442         forest[t] = forest[tip[t]];
443         type[t] = EVEN;
444         root[t] = root[tip[t]];
445         active[t] = true;
446         outer[t] = t;
447         mate[t] = mate[tip[t]];
448
449         return t;
450     }
451
452     void Matching::UpdateDualCosts()
453     {
454         double e1 = 0, e2 = 0, e3 = 0;
455         int inite1 = false, inite2 = false, inite3 = false;
456         for(int i = 0; i < m; i++)
457         {
458             int u = G.GetEdge(i).first,
459                 v = G.GetEdge(i).second;
460
461             if( (type[outer[u]] == EVEN and type[outer[v]] ==
462 UNLABELED) or (type[outer[v]] == EVEN and type[outer[u]] == UNLABELED) )
463             {
464                 if(!inite1 or GREATER(e1, slack[i]))
465                 {
466                     e1 = slack[i];
467                     inite1 = true;
468                 }
469             }
470             else if( (outer[u] != outer[v]) and type[outer[u]] ==
471 EVEN and type[outer[v]] == EVEN )
472             {
473                 if(!inite2 or GREATER(e2, slack[i]))
474                 {
475                     e2 = slack[i];
476                     inite2 = true;
477                 }
478             }
479         }
480         for(int i = n; i < 2*n; i++)

```

```

481     {
482         if(active[i] and i == outer[i] and type[outer[i]] ==
483 ODD and (!inite3 or GREATER(e3, dual[i])))
484         {
485             e3 = dual[i];
486             inite3 = true;
487         }
488     }
489     double e = 0;
490     if(inite1) e = e1;
491     else if(inite2) e = e2;
492     else if(inite3) e = e3;
493
494     if(GREATER(e, e2/2.0) and inite2)
495         e = e2/2.0;
496     if(GREATER(e, e3) and inite3)
497         e = e3;
498
499     for(int i = 0; i < 2*n; i++)
500     {
501         if(i != outer[i]) continue;
502
503         if(active[i] and type[outer[i]] == EVEN)
504         {
505             dual[i] += e;
506         }
507         else if(active[i] and type[outer[i]] == ODD)
508         {
509             dual[i] -= e;
510         }
511     }
512
513     for(int i = 0; i < m; i++)
514     {
515         int u = G.GetEdge(i).first,
516             v = G.GetEdge(i).second;
517
518         if(outer[u] != outer[v])
519         {
520             if(type[outer[u]] == EVEN and type[outer[v]] ==
521 EVEN)
522                 slack[i] -= 2.0*e;
523             else if(type[outer[u]] == ODD and
524 type[outer[v]] == ODD)
525                 slack[i] += 2.0*e;
526             else if( (type[outer[v]] == UNLABELED and
527 type[outer[u]] == EVEN) or (type[outer[u]] == UNLABELED and
528 type[outer[v]] == EVEN) )
529                 slack[i] -= e;
530             else if( (type[outer[v]] == UNLABELED and
531 type[outer[u]] == ODD) or (type[outer[u]] == UNLABELED and
532 type[outer[v]] == ODD) )
533                 slack[i] += e;
534         }
535     }
536     for(int i = n; i < 2*n; i++)
537     {
538         if(GREATER(dual[i], 0))
539         {
540             blocked[i] = true;
541         }
542         else if(active[i] and blocked[i])
543         {
544             //The blossom is becoming unblocked

```

```

545             if(mate[i] == -1)
546             {
547                 DestroyBlossom(i);
548             }
549             else
550             {
551                 blocked[i] = false;
552                 Expand(i);
553             }
554         }
555     }
556 }
557
558 pair< list<int>, double>
559 Matching::SolveMinimumCostPerfectMatching(const vector<double> & cost)
560 {
561     SolveMaximumMatching();
562     if(!perfect)
563         throw "Error: The graph does not have a perfect
564 matching";
565
566     Clear();
567
568     //Initialize slacks (reduced costs for the edges)
569     slack = cost;
570
571     PositiveCosts();
572
573     //If the matching on the compressed graph is perfect, we are
574 done
575     perfect = false;
576     while(not perfect)
577     {
578         //Run an heuristic maximum matching algorithm
579         Heuristic();
580         //Grow a hungarian forest
581         Grow();
582         UpdateDualCosts();
583         //Set up the algorithm for a new grow step
584         Reset();
585     }
586
587     list<int> matching = RetrieveMatching();
588
589     double obj = 0;
590     for(list<int>::iterator it = matching.begin(); it !=
591 matching.end(); it++)
592         obj += cost[*it];
593
594     double dualObj = 0;
595     for(int i = 0; i < 2*n; i++)
596     {
597         if(i < n) dualObj += dual[i];
598         else if(blocked[i]) dualObj += dual[i];
599     }
600
601     return pair< list<int>, double >(matching, obj);
602 }
603
604 void Matching::PositiveCosts()
605 {
606     double minEdge = 0;
607     for(int i = 0; i < m ;i++)
608         if(GREATER(minEdge - slack[i], 0))

```

```

        minEdge = slack[i];

        for(int i = 0; i < m; i++)
            slack[i] -= minEdge;
    }

    list<int> Matching::SolveMaximumMatching()
    {
        Clear();
        Grow();
        return RetrieveMatching();
    }

    //Sets up the algorithm for a new run
    void Matching::Clear()
    {
        ClearBlossomIndices();

        for(int i = 0; i < 2*n; i++)
        {
            outer[i] = i;
            deep[i].clear();
            if(i<n)
                deep[i].push_back(i);
            shallow[i].clear();
            if(i < n) active[i] = true;
            else active[i] = false;

            type[i] = 0;
            forest[i] = -1;
            root[i] = i;

            blocked[i] = false;
            dual[i] = 0;
            mate[i] = -1;
            tip[i] = i;
        }
        slack.assign(m, 0);
    }

    list<int> Matching::RetrieveMatching()
    {
        list<int> matching;

        for(int i = 0; i < 2*n; i++)
            if(active[i] and mate[i] != -1 and outer[i] == i)
                Expand(i, true);

        for(int i = 0; i < m; i++)
        {
            int u = G.GetEdge(i).first;
            int v = G.GetEdge(i).second;

            if(mate[u] == v)
                matching.push_back(i);
        }
        return matching;
    }

```

```

1  #pragma once
2
3  #include "Graph.h"
4  #include "BinaryHeap.h"
5  #include <list>
6  #include <vector>
7  using namespace std;
8
9  #define EVEN 2
10 #define ODD 1
11 #define UNLABELED 0
12
13 class Matching
14 {
15 public:
16     //Parametric constructor receives a graph instance
17     Matching(const Graph & G);
18
19     //Solves the minimum cost perfect matching problem
20     //Receives the a vector whose position i has the cost of the edge
21 with index i
22     //If the graph does not have a perfect matching, a const char *
23 exception will be raised
24     //Returns a pair
25     //the first element of the pair is a list of the indices of the
26 edges in the matching
27     //the second is the cost of the matching
28     pair< list<int>, double > SolveMinimumCostPerfectMatching(const
29 vector<double> & cost);
30
31     //Solves the maximum cardinality matching problem
32     //Returns a list with the indices of the edges in the matching
33     list<int> SolveMaximumMatching();
34
35 private:
36     //Grows an alternating forest
37     void Grow();
38     //Expands a blossom u
39     //If expandBlocked is true, the blossom will be expanded even if
40 it is blocked
41     void Expand(int u, bool expandBlocked);
42     //Augments the matching using the path from u to v in the
43 alternating forest
44     void Augment(int u, int v);
45     //Resets the alternating forest
46     void Reset();
47     //Creates a blossom where the tip is the first common vertex in
48 the paths from u and v in the hungarian forest
49     int Blossom(int u, int v);
50     void UpdateDualCosts();
51     //Resets all data structures
52     void Clear();
53     void DestroyBlossom(int t);
54     //Uses an heuristic algorithm to find the maximum matching of the
55 graph
56     void Heuristic();
57     //Modifies the costs of the graph so the all edges have positive
58 costs
59     void PositiveCosts();
60     list<int> RetrieveMatching();
61
62     int GetFreeBlossomIndex();
63     void AddFreeBlossomIndex(int i);

```

```

void ClearBlossomIndices();

//An edge might be blocked due to the dual costs
bool IsEdgeBlocked(int u, int v);
bool IsEdgeBlocked(int e);
//Returns true if u and v are adjacent in G and not blocked
bool IsAdjacent(int u, int v);

const Graph & G;

list<int> free;//List of free blossom indices
64
65
66 vector<int> outer;//outer[v] gives the index of the outermost
67 blossom that contains v, outer[v] = v if v is not contained in any
68 blossom
69 vector< list<int> > deep;//deep[v] is a list of all the original
70 vertices contained inside v, deep[v] = v if v is an original vertex
71 vector< list<int> > shallow;//shallow[v] is a list of the
72 vertices immediately contained inside v, shallow[v] is empty is the
73 default
74 vector<int> tip;//tip[v] is the tip of blossom v
75 vector<bool> active;//true if a blossom is being used
76
77 vector<int> type;//Even, odd, neither (2, 1, 0)
78 vector<int> forest;//forest[v] gives the father of v in the
79 alternating forest
80 vector<int>
81 root;//root[v] gives the root of v in the alternating forest
82
83 vector<bool> blocked;//A blossom can be blocked due to dual
84 costs, this means that it behaves as if it were an original vertex and
85 cannot be expanded
86 vector<double> dual;//dual multipliers associated to the
87 blossoms, if dual[v] > 0, the blossom is blocked and full
88 vector<double>
89 slack;//slack associated to each edge, if slack[e] > 0, the edge cannot
90 be used
91 vector<int> mate;//mate[v] gives the mate of v
92
93 int m, n;
94
95 bool perfect;
96
97 list<int> forestList;
98 vector<int> visited;
99
100 };

```